Universidad Argentina de la Empresa

Materia: **PROGRAMACIÓN ORIENTADA A OBJETOS**

Trabajo Práctico Obligatorio: Etapa 2 - Informe final



Integrantes del equipo:

- Núñez, Manuel | LU: 1138585
- Vasquez, Victoria | LU: 1218864
- Giammarino, Francisco | LU: 1160277

Docente: León Maria Angela

Repositorio github: https://github.com/vicfabelo/TpProgramacionOO

Introducción

Este documento corresponde a la segunda etapa del trabajo práctico. En esta etapa programamos en Java el sistema que diseñamos en la Etapa 1 para la agencia Rumbo Sur, lo probamos con pruebas unitarias y funcionales, y corregimos los errores que encontramos.

Reúne en un solo lugar todo lo que pide la entrega además del código: las instrucciones de instalación y uso (lo mismo que el README.md del repositorio), las pruebas unitarias y el plan de pruebas funcionales con sus resultados, y el informe final con los cambios respecto del diseño, las dificultades, la distribución de tareas y la reflexión del grupo. El código fuente, el README, las pruebas y la documentación (carpeta docs, con los diagramas actualizados) están en el repositorio del equipo.

1. Sistema entregado

Es una aplicación de escritorio en Java 17 con interfaz gráfica Swing. Implementa los quince casos de uso del documento de diseño: catálogo de tours, programación de salidas, registro de clientes, reservas con cálculo de precio y promociones, pagos, servicios adicionales, cancelaciones, reemplazos y notas de ajuste, asignación de guía y vehículo, lista de pasajeros, inicio y cierre de salidas, liquidación de guías, reportes y control diario de vencimientos.

La ventana tiene cuatro pestañas: *Catálogo y salidas*, *Clientes y reservas*, *Operación* y *Reportes*. Los datos se guardan automáticamente en el archivo datos/agencia.dat después de cada operación y al cerrar la ventana. La primera vez que se ejecuta, el sistema carga datos de prueba (tres tours, cuatro salidas, clientes, personal y reservas) con fechas calculadas a partir del día actual, para poder probar todo enseguida.

2. Instalación y uso (README)

Esta sección reproduce el contenido del archivo README.md que está en la raíz del repositorio.

2.1 Requisitos

- **JDK 17 o superior.**
- **IntelliJ IDEA** (recomendado) o **Maven 3.8+**. Maven descarga JUnit la primera vez, así que hace falta conexión a internet.

2.2 Cómo ejecutarlo

Con IntelliJ IDEA

1. *File → Open* y elegir la carpeta del proyecto (la que tiene pom.xml). Aceptar *Open as Project* / *Trust Project*.
2. Esperar a que IntelliJ termine de importar el proyecto Maven.
3. Abrir src/turismo/main/Main.java y ejecutar con el botón verde de ejecución (*Run*) que aparece junto a main.

Con Maven desde la consola

```
mvn package
java -jar target/agencia-turismo.jar
```

Los datos se guardan en la carpeta datos/, dentro de la carpeta desde donde se ejecuta. Para volver a los datos de prueba iniciales, hay que cerrar el programa y borrar esa carpeta.

2.3 Cómo correr las pruebas

- **IntelliJ:** clic derecho sobre la carpeta test → *Run 'All Tests'*.
- **Maven:** mvn test

Son 77 pruebas unitarias de la lógica de negocio (JUnit 4). El detalle está en la sección 4 y las pruebas funcionales en la sección 5.

2.4 Datos de prueba

La primera vez que se ejecuta, el sistema carga estos datos de ejemplo, con fechas calculadas desde el día actual:

Tipo	Datos
Tours	NAT-01 Parque Nacional Talampaya (naturaleza, 3 días) AVE-01 Rafting en el río Atuel (aventura, edad mínima 14, seguro obligatorio) CUL-01 Caminos del vino en Luján de Cuyo (cultural, 1 día)
Salidas	SAL-001 Talampaya en 20 días (confirmada, con guía y vehículo) SAL-002 Rafting en 40 días (programada) SAL-003 Bodegas en 75 días (confirmada) SAL-004 Talampaya en 110 días (sin reservas)
Personal	Vendedores Laura Gómez y Martín Pereyra Guías Sofía Ruiz (aventura y naturaleza) y Diego Fernández (cultural y naturaleza) Coordinadora Ana Torres · Administrador Carlos Díaz
Clientes	Lucía Méndez, Pablo Souza (Brasil), Juan Ibáñez
Flota	Micro AB123CD (40 lugares), Combi AC456EF (15), Camioneta AD789GH (12)

2.5 Para probar rápido

- **Promoción anticipada:** reservar en SAL-004 (más de 60 días) aplica el 15 %.
- **Promoción grupo:** 6 o más pasajeros en una misma reserva aplica el 10 % (si también corresponde la anticipada, se aplica la mejor).
- **Edad mínima:** un pasajero menor de 14 años en Rafting es rechazado.
- **Operación completa:** en *Operación*, con SAL-001: *Emitir lista de pasajeros → Iniciar salida → Finalizar salida → Liquidar guía*.

2.6 Estructura del repositorio

Carpeta o archivo	Contenido
src/turismo/interfaces	Promocion, PoliticaCancelacion, RepositorioAgencia
src/turismo/clases	Modelo del dominio y enumeraciones
src/turismo/excepciones	Excepciones de negocio y de persistencia

Carpeta o archivo	Contenido
src/turismo/persistencia	Guardado en archivo, exportación a texto y datos de prueba
src/turismo/servicios	Controlador (AgenciaController) y reportes
src/turismo/ui	Interfaz gráfica Swing
src/turismo/main	Main (punto de entrada)
test	Pruebas unitarias JUnit
pom.xml	Configuración de Maven (Java 17, JUnit 4)
docs	Enunciado, documento de diseño de la Etapa 1, este informe y el plan de pruebas
docs/uml	Diagramas en StarUML (.mdj) y exportados como imagen (.png)
README.md	Instrucciones de instalación y uso

2.7 Patrones aplicados

- **SOLID:** responsabilidad única (SRP), abierto/cerrado (OCP) e inversión de dependencias (DIP).
- **GRASP:** Experto en información, Creador, Controlador y Polimorfismo.

La justificación de cada uno está en el documento de diseño de la Etapa 1, sección 5.

3. Cumplimiento de los requisitos técnicos

Requisito	Cómo lo cumplimos	Dónde verlo
Lenguaje y paquetes	Java 17, organizado en siete paquetes: interfaces, clases, excepciones, persistencia, servicios, ui y main.	src/turismo/
Herencia y polimorfismo	Tres jerarquías con métodos sobrescritos: Tour (abstracta) con sus tres categorías; Persona (abstracta) con Cliente, Pasajero y Empleado, que a su vez tiene cuatro roles; y PanelBase (abstracta) con los cuatro paneles. Además, tres interfaces propias con seis implementaciones.	Tour, TourAventura, Persona, Empleado, Promocion, PoliticaCancelacion
Encapsulamiento	Atributos privados con getters; protected en las superclases; constructores que validan los datos; constructor de paquete en Reserva para que solo la cree Salida; listas que se devuelven sin poder modificarse desde afuera.	Persona, Tour, Empleado, Reserva
Colecciones	ArrayList, EnumSet (categorías del guía), EnumMap (devolución por medio de pago), HashMap y LinkedHashMap (reportes).	Guia, Cancelacion, GeneradorReportes
Ordenamiento natural	Actividad se ordena por día y orden; Salida, por fecha de inicio (Comparable).	Actividad.compareTo(), Salida.compareTo()
Ordenamiento personalizado	Lista de pasajeros por apellido y nombre; reportes por importe y por fecha (Comparator).	Pasajero.POR_APELLIDO_Y_NOMBRE, GeneradorReportes

Requisito	Cómo lo cumplimos	Dónde verlo
Excepciones	Excepción propia chequeada ReglaNegocioExcepcion con cuatro subclases, lanzadas con throw y declaradas con throws. Se usa try, catch y finally en la interfaz, en la persistencia y al cerrar la ventana.	excepciones/, RepositorioArchivo, PanelBase
Entrada y salida	Los datos se guardan y se leen de un archivo (serialización, con archivo temporal para no dañar el original). La lista de pasajeros y los reportes se exportan a .txt.	RepositorioArchivo, ExportadorTexto
Interfaz	Interfaz gráfica Swing con pestañas, tablas, formularios y manejo de eventos (ActionListener, ListSelectionListener, ChangeListener, WindowListener).	ui/
Patrones	SRP, OCP y DIP; Experto, Creador, Controlador y Polimorfismo, tal como se justificaron en la Etapa 1.	Documento de diseño, sección 5
Pruebas	77 pruebas unitarias JUnit sobre la lógica de negocio y 22 pruebas funcionales documentadas con su resultado.	test/ y secciones 4 y 5

4. Pruebas unitarias

Las pruebas unitarias son automáticas (JUnit 4) y están en la carpeta test del repositorio. Cubren la lógica de negocio sin abrir la interfaz gráfica: precios por tipo de pasajero, promociones, seña y saldo, recargo por cuotas, políticas de cancelación, cupos, estados de la salida, asignación de guía y vehículo, liquidación, reportes y guardado en archivo.

Para que los resultados no dependan del día en que se ejecutan, todas usan fechas fijas (clase Escenario). AgenciaControllerTest usa además un reloj fijo y un repositorio en memoria que reemplaza al archivo, algo posible porque el controlador depende de la interfaz RepositorioAgencia y no de una clase concreta (DIP). Se ejecutaron por última vez el 09/11/2026.

Clase de prueba	Qué verifica	Tests	Resultado
PasajeroTest	Porcentaje de tarifa por edad y tipo (adulto, menor de 12, menor de 3, jubilado), edad calculada a la fecha de la salida, Comparator por apellido y nombre.	8	OK
TourTest	Itinerario ordenado (Comparable), tarifa por temporada y por vigencia, edad mínima y seguro obligatorio en aventura, cupo de la salida dentro de los límites del tour.	11	OK
ReservaTest	Precio por tipo de pasajero, promociones (la más favorable, sin acumular, no aplica a adicionales), seña del 30 %, recargo del 7 % por cuotas, fecha límite del saldo, políticas de cancelación y devolución por medio de pago, reemplazo de pasajero, notas de ajuste.	25	OK

Clase de prueba	Qué verifica	Tests	Resultado
SalidaTest	Cupo excedido, confirmación por cupo mínimo, salida completa, control a 7 días, cancelación de la agencia, guía habilitado y sin superposición, capacidad del vehículo, inicio sin guía o sin lista, lista ordenada, liquidación del guía.	16	OK
AgenciaControllerTest	Primera ejecución con datos de prueba, numeración correlativa de reservas y salidas aunque falle una, salidas en el pasado, documento duplicado, política según quién cancela, vencimientos, vehículo superpuesto, lista de pasajeros en archivo, exportación de reportes.	10	OK
GeneradorReportesTest	Ventas por vendedor ordenadas sin anuladas, ingresos netos de devoluciones, reporte de cancelaciones, conversión a texto.	4	OK
RepositorioArchivoTest	Guardar y volver a leer los datos (con las referencias entre objetos), archivo inexistente, archivo dañado.	3	OK
Total		**77**	**77 OK**

5. Pruebas funcionales de los casos de uso principales

Las pruebas funcionales se hicieron sobre los datos de prueba iniciales del sistema (los que se cargan la primera vez que se abre el programa), con la fecha del sistema fija en 15/10/2026. Para cada caso se registró el dato de entrada, el resultado esperado y el resultado obtenido.

ID	Caso de uso	Dato de entrada	Resultado esperado	Resultado obtenido	Estado
F01	CU01/ CU02	Tour de aventura "Escalada" con tarifa pero sin seguro; se programa una salida.	Rechazo: los tours de aventura deben ofrecer seguro.	"El tour de aventura Escalada debe ofrecer seguro de viaje".	OK
F02	CU02	Tour NAT-01 (3 días), fecha 14/11/2026, cupo 10.	Salida PROGRAMADA del 14/11 al 16/11 con código correlativo.	SAL-005 PROGRAMADA del 14/11/2026 al 16/11/2026 (el intento fallido de F01 no consumió el código).	OK
F03	CU02	Tour NAT-01 (cupo 5 a 15) con cupo 20.	Rechazo por cupo fuera de los límites.	"El cupo de la salida debe estar entre 5 y 15 (límites del tour)".	OK
F04	CU03	Cliente con documento 35111222 (ya existe).	Rechazo por documento repetido.	"Ya existe un cliente con documento 35111222".	OK
F05	CU04/ CU05	2 adultos en SAL-004 (febrero = temporada alta, $260.000), reservando 110 días antes.	Reserva PENDIENTE con promoción "Reserva anticipada": 2 × 260.000 × 0,85 = $442.000.	Reserva 5 PENDIENTE, promoción "Reserva anticipada", total $442.000 (tarifas $520.000).	OK

ID	Caso de uso	Dato de entrada	Resultado esperado	Resultado obtenido	Estado
F06	CU04	10 pasajeros en SAL-002 (cupo 12, 3 ocupados).	CupoExcedidoExcepcion indicando los lugares libres.	"Se piden 10 lugares y quedan 9 en la salida SAL-002".	OK
F07	CU04	Pasajero de 11 años en Rafting (edad mínima 14).	PasajeroNoValidoExcepcion.	"Nene, P tiene 11 años y el tour Rafting en el río Atuel exige 14".	OK
F08	CU06	Seña de la reserva 3 (30 % de $447.000) por transferencia.	Reserva CONFIRMADA; saldo $312.900.	Reserva 3 CONFIRMADA, pagado $134.100, saldo $312.900.	OK
F09	CU06	Pago de $100.000 con tarjeta en 3 cuotas.	Recargo 7 % ($7.000) cobrado aparte; el saldo baja $100.000.	Recargo $7.000, cobrado $107.000, saldo $212.900.	OK
F10	CU07	Contratar "Excursión nocturna de estrellas" ($25.000) a un pasajero de la reserva 1.	El total sube $25.000.	Total $525.000 → $550.000.	OK
F11	CU08	Reserva en SAL-003 (faltan 75 días), pagado $50.000 en efectivo; cancela el cliente.	Devolución 90 % = $45.000 en efectivo.	90 % = $45.000 por EFECTIVO.	OK
F12	CU08	Reserva con $40.000 pagados con tarjeta; cancela la agencia.	Devolución 100 % = $40.000.	100 % = $40.000.	OK
F13	CU09	Reemplazar a "Ruiz, Elena" en la reserva 1 (faltan 20 días).	Reemplazo aceptado sin cambiar el total.	Reemplazo aceptado, total sin cambios $550.000.	OK
F14	CU10	Asignar a Diego Fernández (cultural y naturaleza) a la salida de rafting (aventura).	GuiaNoDisponibleExcepcion.	"Fernández, Diego no está habilitado para tours de AVENTURA".	OK
F15	CU10	Asignar la camioneta de 12 lugares a SAL-001 (cupo 15).	Rechazo por capacidad.	"El vehículo AD789GH tiene 12 lugares y la salida tiene cupo 15".	OK
F16	CU12	Iniciar SAL-001 sin haber emitido la lista.	EstadoInvalidoExcepcion.	"No se puede iniciar la salida SAL-001 sin emitir la lista de pasajeros".	OK
F17	CU11/ CU12	Emitir la lista de SAL-001, iniciar y finalizar.	Archivo con 6 pasajeros; estado EN_CURSO y luego FINALIZADA.	lista_SAL-001.txt (6 pasajeros); EN_CURSO → FINALIZADA.	OK
F18	CU13	Liquidar a Sofía Ruiz por SAL-001 (3 días × $45.000; adicionales vendidos $25.000).	$135.000 + 5 % de $25.000 = $136.250.	3 × $45.000 + 5 % de $25.000 = $136.250.	OK
F19	CU14	Reporte de ventas por vendedor.	Vendedores ordenados de mayor a menor venta, sin reservas anuladas.	Gómez, Laura $1.439.000; Pereyra, Martín $1.083.000.	OK

ID	Caso de uso	Dato de entrada	Resultado esperado	Resultado obtenido	Estado
F20	CU15	Procesar vencimientos 8 días antes de SAL-002 (reserva 3 con saldo).	Reserva 3 anulada por saldo impago (sin devolución); SAL-002 todavía no se cancela.	Reserva 3 anulada por saldo impago; SAL-002 sigue (también se canceló SAL-005, que ya estaba a menos de 7 días sin cupo mínimo).	OK
F21	CU15	Procesar vencimientos 7 días antes de SAL-002.	SAL-002 cancelada por no alcanzar el cupo mínimo, con devolución del 100 %.	SAL-002 cancelada por no alcanzar el cupo mínimo (devolución 100 %).	OK
F22	Persistencia	Guardar, cerrar y volver a abrir el sistema.	Se recuperan todas las salidas y reservas con su estado.	5 salidas y 7 reservas recuperadas; SAL-001 sigue FINALIZADA.	OK

5.1 Error encontrado y corregido

En la primera ejecución, F02 devolvió **SAL-006** en lugar de SAL-005: el intento fallido de F01 había consumido un código de salida. Se corrigió en DatosAgencia y AgenciaController.programarSalida() (el código se confirma recién cuando la salida se crea bien, igual que el número de reserva) y se agregó la prueba unitaria losCodigosDeSalidaNoSeSalteanAunqueFalle. Después de la corrección todas las pruebas dieron OK.

5.2 Prueba de la interfaz gráfica

Se abrió la aplicación y se recorrieron las cuatro pestañas: las tablas muestran los datos de prueba y al seleccionar un tour o una reserva se ve su detalle. Además se repitieron a mano desde la interfaz los casos de error, para verificar que el mensaje aparece en un cuadro de advertencia sin cerrar el programa (lo maneja PanelBase.ejecutar()):

ID	Acción en la interfaz	Resultado obtenido	Estado
F04	Clientes y reservas → Registrar cliente con documento 35111222.	Se muestra cuadro de advertencia: "Ya existe un cliente con documento 35111222". El cliente no se registra.	OK
F07	Clientes y reservas → Nueva reserva en Rafting con un pasajero de 11 años (Prueba, Nene).	Se muestra cuadro de advertencia: "Prueba, Nene tiene 11 años y el tour Rafting en el río Atuel exige 14". La reserva no se crea.	OK
F15	Operación → Asignar la camioneta AD789GH (12 lugares) a SAL-001 (cupo 15).	Se muestra cuadro de advertencia: "El vehículo AD789GH tiene 12 lugares y la salida tiene cupo 15".	OK
F16	Operación → Iniciar SAL-001 sin emitir la lista de pasajeros.	Se muestra cuadro de advertencia: "No se puede iniciar la salida SAL-001 sin emitir la lista de pasajeros".	OK

6. Cambios respecto del diseño original

Durante la programación tuvimos que hacer algunos cambios al diseño de la Etapa 1. Los resumimos en la siguiente tabla junto con el motivo de cada uno.

Cambio	Motivo
Se agregó la clase Formulario en el paquete ui, usada por PanelBase y los cuatro paneles.	Todas las pantallas piden datos con cuadros de diálogo parecidos. Para no repetir ese código en cada panel, una sola clase arma los campos y valida lo que se ingresa (números, fechas, campos obligatorios). Se agregó al diagrama de clases final.
Pasajero tiene el Comparator estático POR_APELLIDO_Y_NOMBRE.	La lista de pasajeros tiene que salir ordenada por apellido y nombre, y ese no es el orden natural de ningún objeto. Se agregó al diagrama de clases final como atributo estático de Pasajero.
El número de reserva y el código de salida se obtienen en dos pasos (consultar y confirmar).	Si la creación falla, el número no se pierde y no quedan números salteados.
El control de que un vehículo no esté en dos salidas a la vez se hace en el controlador.	Para controlarlo hay que mirar todas las salidas, y esa información la tiene el controlador, no cada salida.
Los nombres de los archivos exportados se guardan sin tildes.	Una prueba falló al exportar un reporte con tilde en el nombre en un sistema sin codificación UTF-8.
Las excepciones propias se nombraron en español (ReglaNegocioExcepcion, CupoExcedidoExcepcion, etc.).	Para que todos los nombres del código estén en el mismo idioma. Los diagramas se actualizaron con los nombres nuevos.
Correcciones en los diagramas: en el de casos de uso, CU04 incluye a CU05 («include») y CU07 extiende a CU04 («extend»); en el de clases final, las interfaces se muestran con sus métodos.	Los diagramas de la Etapa 1 tenían esas relaciones mal marcadas. Los diagramas corregidos están en la carpeta docs/uml del repositorio.

7. Dificultades encontradas

- **Las fechas en las reglas de negocio.** Casi todas las reglas dependen del día de hoy. Al principio las pruebas daban distinto según el día en que se corrían. Lo resolvimos pasando el reloj al controlador y usando fechas fijas en las pruebas.
- **Las referencias cruzadas al guardar.** Salida conoce sus reservas y cada Reserva conoce su salida; Guia conoce sus salidas. La serialización de Java respeta esas referencias, pero tuvimos que asegurarnos de que todas las clases fueran Serializable y comprobar con RepositorioArchivoTest que, al volver a leer el archivo, los objetos siguen relacionados igual.
- **El estado de la salida.** Definir cuándo una salida pasa a confirmada o a completa, y qué pasa si después se cancela una reserva, nos llevó a definir dos reglas que el enunciado no aclara: para el cupo mínimo solo cuentan los pasajeros de reservas

con seña pagada, y si una salida completa libera lugares por una cancelación, vuelve a estar confirmada.

- **Números salteados.** Las pruebas funcionales mostraron que un intento fallido de programar una salida consumía un código. Lo corregimos y agregamos una prueba para que no vuelva a pasar.
- **Organización del equipo.** Coordinar los horarios de los tres no fue fácil, y además era la primera vez que trabajábamos con Git en un proyecto compartido. Al principio se subieron clases que no seguían el diagrama de la Etapa 1 (faltaba la superclase Persona, aparecía una clase Itinerario que no estaba en el diseño y el controlador quedó en el paquete equivocado), y hubo que corregirlas para que el código coincidiera con lo entregado. También tuvimos problemas para autenticarnos al hacer push y con el proyecto guardado dentro de una carpeta sincronizada de OneDrive, que terminamos moviendo a otra ubicación. A partir de ahí acordamos subir el código de a pocas clases por commit, verificando cada vez que compilara.

8. Distribución de tareas

La tabla resume lo que hizo cada integrante en la Etapa 2, tal como queda registrado en los commits del repositorio.

Integrante	Tareas	Commits y archivos principales
Núñez, Manuel	Configuración del proyecto Maven. Programación de la mayor parte del sistema: excepciones, persistencia, enumeraciones, jerarquía de personas, tours, salidas, reservas, pagos, cancelaciones, promociones, controlador, reportes, datos de prueba e interfaz gráfica. Corrección de las clases que no coincidían con el diagrama. Las 77 pruebas unitarias, el plan de pruebas funcionales, la actualización de los diagramas y este informe.	src/turismo/ (todos los paquetes), test/, docs/, pom.xml, README.md
Vasquez, Victoria		
Giammarino, Francisco		

9. Reflexión del grupo

Lo que más nos sirvió fue haber pensado bien el diseño antes de programar. Al aplicar los patrones en la Etapa 1, la clase Agencia que hacía todo se dividió en varias clases con una sola tarea, y eso hizo que pudiéramos programar el sistema por partes, de a pocas clases por vez, sin perder la coherencia con el diagrama.

Las interfaces Promocion y PoliticaCancelacion mostraron su utilidad durante el desarrollo: cuando tuvimos que agregar la anulación por falta de pago, alcanzó con crear una clase nueva, sin tocar Reserva. También aprendimos lo importante que es separar la interfaz gráfica de la lógica: como las pantallas solo llaman al controlador, pudimos probar todos los casos de uso con JUnit sin abrir ninguna ventana.

Como equipo, aprendimos que repartir el trabajo no alcanza: antes de programar hay que acordar cómo se va a hacer (nombres de clases, paquetes y relaciones del diagrama) y revisar lo que sube cada uno, porque las clases que no seguían el diseño nos obligaron a rehacer trabajo. También nos dimos cuenta de que el diagrama de clases es útil como contrato entre los integrantes: cuando hubo dudas, lo resolvimos volviendo a él.

Si volviéramos a empezar, repartiríamos las tareas de forma más eficiente desde el principio, con fechas intermedias para cada parte, y escribiríamos las pruebas a medida que programamos y no al final, porque varios errores los encontramos recién al probar.